

PYTHON 3

- Le module Networkx -

On supposera importés les modules ci-dessous :

```
import numpy as np
```

Pour utiliser les commandes Python permettant de travailler sur les graphes, on commencera par importer le module `networkx` sous la forme :

```
import networkx as nx
```

On présente ici quelques commandes du module `networkx`. Pour plus de détails, vous pouvez vous rendre ici : <https://networkx.org/>

Définir un graphe en Python avec networkx

Pour définir un graphe non orienté nommé G , on utilise la commande

```
G=nx.Graph(***)
```

et pour définir un graphe non orienté nommé G , on utilise la commande

```
G=nx.DiGraph(***)
```

avec pour argument (représenté ici par les `***`) :

- Soit la liste des arêtes (resp. arcs) donnée sous la forme de couples donnant le sommet d'origine et l'extrémité de l'arête (ou de l'arc).

Notons que si le graphe n'est pas orienté, pour définir l'arête  le couple ('A', 'B') suffit.

Si le graphe est orienté, les couples ('A', 'B') et ('B', 'A') ne définissent pas les mêmes arcs.

- Soit la matrice d'adjacence définissant le graphe, et dans ce cas Python nomme les sommets du graphe $0, 1, \dots, n-1$, en notant n l'ordre du graphe.

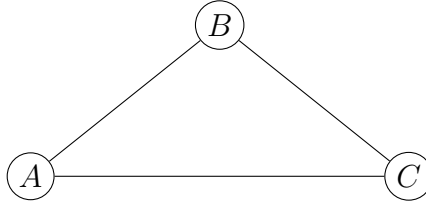
Si le graphe d'origine a ses sommets nommés par des chaînes de caractères, ou par une numérotation différente, nous pouvons les renommer en Python à l'aide de la commande `nx.relabel_nodes(G, ***)`, où `***` est un dictionnaire, chaque clé étant l'ancien nom du sommet et la valeur associée à une clé est le nouveau nom que l'on souhaite attribuer au sommet. Par exemple, pour renommer les trois premiers sommets du graphe G sous la forme A, B et C , il suffit d'écrire :

```
nouveaux_noms = {0: 'A', 1: 'B', 2: 'C'}  
G = nx.relabel_nodes(G, nouveaux_noms)
```

- Soit un dictionnaire définissant le graphe.

Exemple :

Pour définir en Python le graphe non orienté ci-dessous



on peut utiliser l'une des trois méthodes qui viennent d'être présentées :

(1) Méthode 1 : Liste des arêtes

```
G=nx.Graph([('A','B'),('A','C'),('B','C')])
```

(2) Méthode 2 : Matrice d'adjacence

```
M=np.array([[0,1,1],[1,0,1],[1,1,0]])
G=nx.Graph(M)
noms={0:'A',1:'B',2:'C'}
G=nx.relabel_nodes(G,noms)
```

(3) Méthode 3 : Dictionnaire

```
D={'A':['B','C'],'B':['A','C'],'C':['A','B']}
G=nx.Graph(D)
```

Tracé d'un graphe en Python

Pour visualiser un graphe nommé G prédéfini en Python, il ne reste qu'à taper les commandes :

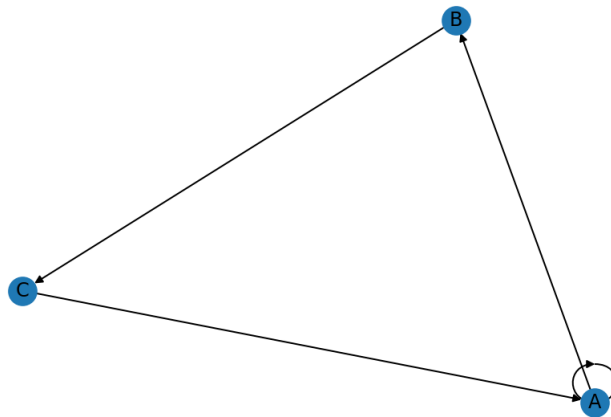
```
nx.draw(G, with_labels=True)
# l'option with_labels=True
# permet d'afficher le nom de chaque sommet
```

Exemple :

Les commandes ci-dessous

```
D={'A':['A','B'],'B':['C'],'C':['A']}
G=nx.DiGraph(D)
nx.draw(G,with_labels=True)
```

affiche, après exécution, la fenêtre graphique suivante :



Quelques fonctions du module networkx

On suppose que G est une variable Python, définissant un graphe (orienté ou non).

La commande Python	renvoie :
<code>G.number_of_nodes()</code>	le nombre de sommets de G
<code>G.number_of_edges()</code>	le nombre d'arêtes (ou d'arcs) de G
<code>list(G.nodes())</code>	la liste des sommets de G
<code>list(G.edges())</code>	la liste des arêtes (ou arcs) de G
<code>list(G.adj[s])</code> où s est l'un des sommets de G	la liste des sommets adjacents (ou la liste des successeurs) du sommet s

Une option graphique

On suppose que G est une variable Python, définissant un graphe (orienté ou non).

La commande Python	permet de disposer les sommets du graphes :
<code>position = nx.circular_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	en cercle
<code>position=nx.bipartite_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	sur deux rangées
<code>position=nx.multipartite_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	sur plusieurs rangées
<code>position=nx.planar_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	sans arêtes (ou arcs) se croisant, lorsque c'est possible
<code>position=nx.random_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	au hasard